zsh Variables Quick Reference

Quoting · arithmetic · parameter expansion · flags · special parameters

MANTRA
"quote it"

QUOTING

`"$var"`	Expands; preserves spaces & newlines. **Default choice.**
`'literal'`	No expansion at all. No way to escape `'` inside (use `'\''`).
`$(cmd)`	Command substitution — nests cleanly, preferred form.
`` `cmd` ``	Legacy backticks — avoid; nesting requires escaping.
`${var}`	Braces delimit the name: `${n}px` vs `$npx`.
`$'a\tb'`	ANSI-C quoting: \t \n \\ \x41 escapes honored.

SPECIAL PARAMETERS

`$0`	Script / shell name
`$1..$9 ${10}`	Positional args (braces ≥ 10)
`$#`	Argument count
`$@` / `$*`	All args (`"$@"` keeps them separate!)
`$?`	Last exit status
`$$` / `$!`	Shell PID / last background PID
`$_`	Last arg of previous command
`$-` / `$TTY`	Option flags / terminal device

ARITHMETIC

`(( n = x + 2 ))`	C-style math; no `$` needed on names
`(( n++ ))` `(( n += 3 ))`	Increment / compound assignment
`echo $(( 2**10 ))`	Expansion form → 1024
`(( n > 5 )) && …`	Exit status 0 when true — use in `if`
`let "n = n + 1"`	Older builtin; same engine
`expr $n + 1`	External, POSIX; slow, word-splitting traps — avoid
`echo 'scale=4; 1/3' | bc`	Arbitrary-precision decimals via pipe
`echo '3 k 1 3 / p' | dc`	RPN cousin of bc
`float f=1.5; (( f * 2 ))`	zsh floats: declare with `typeset -F`
`echo $(( [#16] 255 ))`	Base output → `16#FF`; input `$((16#FF))`

PARAMETER EXPANSION ${…}

`${var:-default}`	Use default if unset/empty
`${var:=default}`	…and also assign it
`${var:+alt}`	Use alt only if var is set
`${var:?msg}`	Abort with msg if unset
`${var%pat}` `${var%%pat}`	Strip shortest/longest suffix
`${var#pat}` `${var##pat}`	Strip shortest/longest prefix
`${var/pat/rep}` `${var//pat/rep}`	Replace first / all matches
`${var:off:len}`	Substring (0-based; neg = from end)
`${#var}`	Length (chars); `${#arr}` = elements
`${var:u}` `${var:l}` `${var:c}`	UPPER / lower / Capitalized (zsh)
`${(q)var}` `${(Q)var}`	Quote-escape / strip one quote level
`${name-$var}`	Indirect: value of var named by $var

EXPANSION FLAGS (zsh-only)

`${(s:,:)v}`	Split on `:` into array
`${(j:,:)a}`	Join array with `:`
`${(f)v}`	Split at newlines
`${(u)a}`	Uniq array elements
`${(o)a}` `${(O)a}`	Sort asc / desc
`${(k)h}` `${(v)h}`	Hash keys / values
`${(P)n}`	Indirect expansion
`${(t)v}`	Variable's type tag

ARRAYS & HASHES

`a=(one two three)`	Indexed array — **1-based** in zsh!
`${a[1]}` `${a[-1]}`	First / last element
`${a[2,4]}`	Slice (inclusive range)
`a+=(four)`	Append element
`typeset -A h; h=(k v)`	Associative array (hash)
`${h[k]}` `${(k)h}`	Value by key / all keys
`${a:#pat}` `${a:|b}`	Remove matches / array difference
`${a[(i)val]}`	Index of first match

DECLARATION & TYPES

`typeset -i n=5`	Integer (auto-eval: `n=4+1` → 5)
`typeset -F f=1.5`	Floating point
`typeset -r PI=3`	Read-only constant
`typeset -x PATH…`	Exported (`export` alias)
`typeset -l x` `-u y`	Force lowercase / UPPERCASE
`local v` / `private v`	Function scope / true privacy
`typeset -T D d ':'`	Tie scalar ↔ array (like PATH/path)

TESTS & CONDITIONS

`[[ -n $v ]]` / `[[ -z $v ]]`	Non-empty / empty
`[[ $a = pat* ]]`	Glob pattern match
`[[ $a =~ re ]]`	Regex; groups in `$match` `$MBEGIN`
`[[ -v name ]]`	Variable is set (zsh 5.3+)
`(( $#ary ))`	Truthy if array non-empty
`[[ $n -eq 3 ]]`	Numeric compare: eq ne lt gt le ge
`${+commands[git]}`	1 if command exists

HANDY TIPS

- zsh does **not** word-split unquoted `$var` by default — but empty vars still vanish. Quote anyway.
- `${var:?}` at the top of a script = instant "required argument" validation.
- Chase full docs interactively: `man zshexpn` — the expansion section is the crown jewel.
- Modifiers chain: `${${(f)"$(cat f)"}:#\#*}` strips comment lines from a file.
- `setopt EXTENDED_GLOB` unlocks `^pat`, `pat#`, `pat##` in patterns.
- Colon-separated vars already tied: `path` ↔ `PATH`, `fpath` ↔ `FPATH`.
- Debug expansion: `print -r -- ${(qqq)var}` shows the value as zsh sees it.
- Need bash-compatible splitting in a pinch? `setopt SH_WORD_SPLIT` locally.

zsh parameter expansion reference · ${…} forms assume zsh 5.x · print at 100% scale